

Raport z wykonanej pracy

Analiza bezpieczeństwa systemu **BEKO Pager Network**

pager-rtos — sieć pagerowa LoRa

Zespół projektowy
15 czerwca 2026

Spis treści

1	Informacje o projekcie	2
1.1	Adresacja i role	2
1.2	Ramka LAVIET_FRAME_V1	2
1.3	Założenia bezpieczeństwa (wg dokumentacji)	2
2	Wykonana praca	2
2.1	Stanowisko badawcze	2
2.2	Odzyskanie dostępu do bramki (Raspberry Pi Gateway)	3
2.3	Weryfikacja materiału kluczowego	4
2.4	Zidentyfikowane podatności	4
2.5	Analiza ostatniej komunikacji (log TX) i podrobienie ramki	5
2.6	Analiza i ataki w warstwie radiowej (SDR)	8
2.7	Demodulacja przechwyconych ramek (LoRa/CSS)	11
3	Rekomendacje naprawcze	12
4	Propozycja implementacji RFID	12

1. Informacje o projekcie

BEKO Pager Network (`pager-rtos`) to bezprzewodowy system pagerowy pracujący w topologii gwiazdy. Węzłem centralnym jest **Raspberry Pi Gateway**, a urządzeniami końcowymi są node'y oparte na mikrokontrolerze **STM32U545**. Komunikacja radiowa realizowana jest przez backend **SX1276/RFM95** w paśmie 868 MHz (LoRa). Gateway wysyła wiadomości do wybranego node'a lub do całej sieci (broadcast), a każda poprawnie odebrana wiadomość powinna zostać potwierdzona ramką **ACK**. Firmware node'a przechowuje do 15 zaufanych urządzeń w lokalnych slotach *trusted storage*.

1.1 Adresacja i role

Adresacja w ramce `LAVIET_FRAME_V1` jest 16-bitowa: `0x0001` to gateway, `0x0002-0xFFFFE` to node'y, a `0xFFFF` oznacza broadcast. W systemie przewidziano cztery role: **Administrator** (pełny dostęp, PIN administratora), **Operator** (ograniczony dostęp administracyjny), **Użytkownik** (podstawowe funkcje node'a) oraz **Serwisant** (fizyczny dostęp diagnostyczny przez UART). Dostęp do lokalnej konfiguracji node'a chroniony jest 4-cyfrowym PIN-em, który zgodnie z dokumentacją nie może być przechowywany jawnie.

1.2 Ramka `LAVIET_FRAME_V1`

System wykorzystuje zwartą ramkę aplikacyjną o strukturze: `ver_type` (1B) | `flags` (1B) | `src_id` (2B) | `dst_id` (2B) | `msg_id` (2B) | `counter` (4B) | `payload_len` (1B) | `payload` (0-16B) | `mac_tag` (32B). Część stała zajmuje 45 B, a całkowity rozmiar ramki mieści się w zakresie 45–61 B (limit 64 B). Pole `ver_type` koduje wersję protokołu (starsze 4 bity) i typ wiadomości (młodsze 4 bity, m.in. `DATA`, `ACK`, `PAIR_REQ`, `KEY_ROTATE`). Pole `flags` zawiera m.in. bity `ENCRYPTED`, `ACK_REQUIRED`, `BROADCAST` i `KEY_UPDATE`.

1.3 Założenia bezpieczeństwa (wg dokumentacji)

Dokumentacja zakłada ochronę poufności, integralności i autentyczności komunikacji:

- **HMAC-SHA256** — każda ramka zawiera 32-bajtowy `mac_tag` liczony po całym nagłówku i payloadzie.
- **AES-CTR** — szyfrowanie payloadu (bez paddingu, długość ciphertextu równa plaintextowi).
- **TPM** — główny punkt zaufania: przechowuje sekret główny urządzenia, chroni klucze i wspiera rotację root seeda (zapis do TPM NV wymaga aktywnej linii TPM PP).
- **Licznik anti-replay** (`counter`, 4 B) — monotoniczny, sprawdzany po stronie odbiorcy.
- **Parowanie** wyłącznie z gatewayem oraz **rotacja kluczy** (docelowo Diffie–Hellman) — radiowy protokół `KEY_ROTATE` pozostaje do dokończenia.

Parametry warstwy radiowej potwierdzone w logach startowych firmware: **868.5 MHz**, `bw=9` (500 kHz), **SF7**, coding rate **4/5**, preambula 8 symboli, sync word `0x34`, CRC ON, nagłówek explicit, IQ normal.

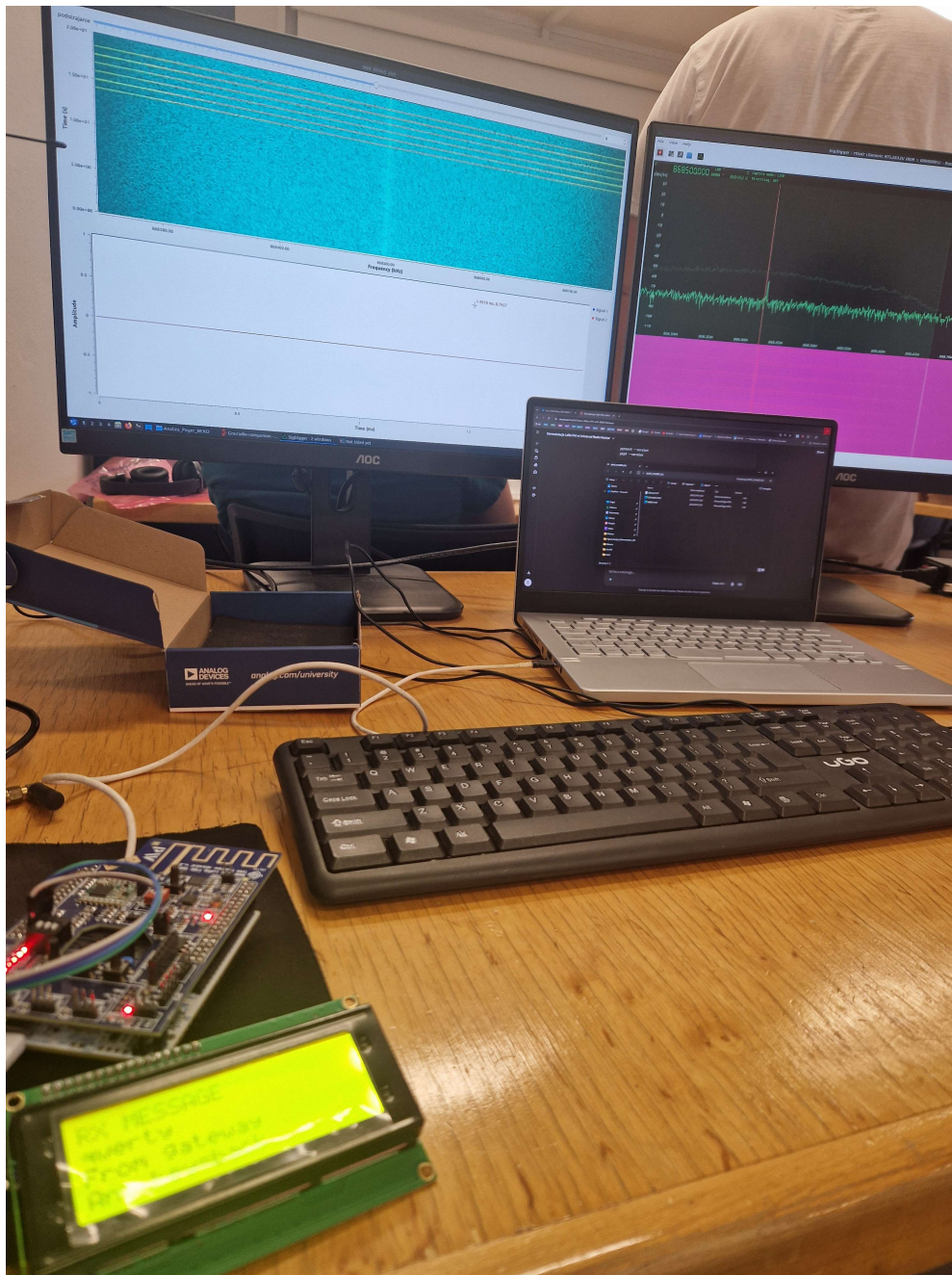
2. Wykonana praca

Celem prac zespołu była praktyczna weryfikacja realnego poziomu bezpieczeństwa systemu względem założeń dokumentacji. Poniżej przedstawiono kolejne etapy przeprowadzonych testów wraz z dowodami.

2.1 Stanowisko badawcze

Testy prowadzono na fizycznym stanowisku obejmującym: node STM32 z wyświetlaczem LCD (pokazujący odebraną wiadomość `RX MESSAGE` `qwerty from gateway`), bramkę z panelem we-

bowym, odbiornik **RTL-SDR** (analiza w SigDigger) oraz nadajnik **ADALM-PLUTO** (**PlutoSDR**, Analog Devices) wykorzystany do generowania sygnałów zakłócających i odtwarzania (replay).



Rysunek 1: Stanowisko badawcze: node STM32 z LCD, RTL-SDR (SigDigger), nadajnik PlutoSDR oraz panele bramki na monitorach.

2.2 Odzyskanie dostępu do bramki (Raspberry Pi Gateway)

W ramach symulacji ataku z fizycznym dostępem odzyskano pełną kontrolę nad bramką Raspberry Pi mimo nieznaności hasła. Kartę SD podłączono do WSL2 przy użyciu narzędzia `usbipd-win`, zamontowano partycję `rootfs` i zmodyfikowano plik `/etc/shadow`. Ponieważ puste hasło było blokowane przez SSH/PAM (`UsePAM yes`), wygenerowano nowy hash poleceniem `openssl passwd -6` i ręcznie wpisano go do `/etc/shadow`, ustawiając nowe poświadczenia logowania (`user / nowehaslo`).

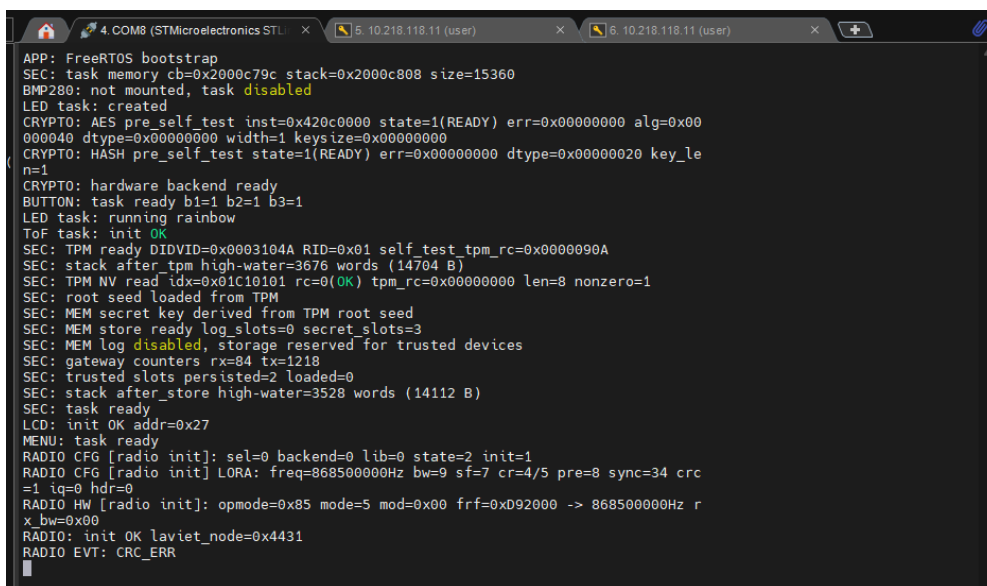
Wniosek: posiadając fizyczny dostęp do nośnika urządzenia, można przejąć pełną kontrolę nad systemem bez znajomości jakiegokolwiek hasła. Karta SD Raspberry Pi domyślnie nie jest szyfrowana — skuteczną ochroną byłoby szyfrowanie nośnika (LUKS), wyłączenie logowania hasłem na rzecz kluczy SSH oraz fizyczne zabezpieczenie urządzenia.

Wpływ na sieć. Przejęcie uprawnień administratora na bramce Raspberry Pi oznacza **całkowitą kompromitację sieci**. Atakujący zyskuje dostęp do bazy zaufanych węzłów, może odczytać lokalne sekrety bramki oraz uzyskuje możliwość wysyłania ramek COUNTER_SYNC z flagą COUNTER_OVERRIDE. Pozwala to na resetowanie liczników w węzłach końcowych, co otwiera drogę do klasycznych ataków **replay** (odtwarzania starych wiadomości) — mimo stosowania liczników anti-replay w protokole.

2.3 Weryfikacja materiału kluczowego

Porównanie logów UART potwierdza, że klucz **HMAC** jest stały (statyczny) dla bieżącej sesji parowania urządzenia. Klucz pozostał niezmieniony po restarcie urządzenia oraz po odebraniu nowej wiadomości typu DATA.

hmac_key: 82 52 30 E0 2F 4B 72 A3 AA AA 6A C1 9C F3 83 5A B8 D2 6D CF 80 1A 22 B3 D6 EE 04 53 86 86 33 A4
--



```
APP: FreeRTOS bootstrap
SEC: task memory cb=0x2000c79c stack=0x2000c808 size=15360
BMP280: not mounted, task disabled
LED task: created
CRYPTO: AES pre_self_test inst=0x420c0000 state=1(READY) err=0x00000000 alg=0x00
000040 dtype=0x00000000 width=1 keysize=0x00000000
CRYPTO: HASH pre_self_test state=1(READY) err=0x00000000 dtype=0x00000020 key_le
n=1
CRYPTO: hardware backend ready
BUTTON: task ready b1=1 b2=1 b3=1
LED task: running rainbow
ToF task: init OK
SEC: TPM ready DIDVID=0x0003104A RID=0x01 self_test_tpm_rc=0x0000090A
SEC: stack after_tpm high-water=3676 words (14704 B)
SEC: TPM NV read idx=0x01C10101 rc=0(OK) tpm_rc=0x00000000 len=8 nonzero=1
SEC: root seed loaded from TPM
SEC: MEM secret key derived from TPM root seed
SEC: MEM store ready log_slots=0 secret_slots=3
SEC: MEM log disabled, storage reserved for trusted devices
SEC: gateway counters rx=84 tx=1218
SEC: trusted slots persisted=2 loaded=0
SEC: stack after_store high-water=3528 words (14112 B)
SEC: task ready
LCD: init OK addr=0x27
MENU: task ready
RADIO CFG [radio init]: sel=0 backend=0 lib=0 state=2 init=1
RADIO CFG [radio init] LORA: freq=868500000Hz bw=9 sf=7 cr=4/5 pre=8 sync=34 crc
=1 iq=0 hdr=0
RADIO HW [radio init]: opmode=0x85 mode=5 mod=0x00 frf=0x092000 -> 868500000Hz r
x_bw=0x00
RADIO: init OK laviet_node=0x4431
RADIO EVT: CRC_ERR
```

Rysunek 2: Logi UART node’a STM32 (FreeRTOS bootstrap, CRYPTO/TPM/RADIO) — źródło materiału kluczowego: klucz `hmac_key` widoczny w logach.

2.4 Zidentyfikowane podatności

Krytyczne naruszenie poufności (logi debugowe). System w wersji produkcyjnej wypisuje na porcie UART surowe dane kryptograficzne, w tym klucz `hmac_key` oraz `pair_code`. Jest to niezgodne z wymogiem ochrony kluczy (sekcje 7 i 8 dokumentacji), zgodnie z którymi klucze nie mogą być przechowywane ani ujawniane w postaci jawnej.

Brak ochrony przed analizą fizyczną (RDP Level 0). Odczyt pamięci FLASH (zrzut `zrzut.bin`) potwierdził brak włączonej ochrony przed odczytem (**RDP Level 0**), co umożliwia inżynierię wsteczną firmware’u i ekstrakcję dalszych sekretów.

Potencjalny wektor: zrzut pamięci RAM przez debugger. Mimo że główny klucz pozyskano z logów UART, brak blokady RDP (Level 0) stanowił równie krytyczne zagrożenie. Umożliwia on podłączenie debuggera sprzętowego (np. **ST-LINK**) w trybie *hot-plug* do działającego mikrokontrolera. Ponieważ TPM musi w czasie pracy wyprowadzić klucz AES/HMAC do pamięci **SRAM** w celu wykonania operacji kryptograficznych na ramce, atakujący mógłby wykonać dynamiczny zrzut RAM i wyekstrahować ulotny materiał kluczowy bezpośrednio z pamięci operacyjnej — nawet gdyby logi UART były wyłączone.

Brak mechanizmów anti-tamper. Podczas fizycznego dostępu do urządzeń nie zaobserwowano żadnych aktywnych mechanizmów zapobiegających manipulacjom (*anti-tamper*). Zgodnie z dobrymi praktykami bezpieczeństwa sprzętowego (m.in. OWASP — kategoria *Physical Interface*) node STM32 powinien wykorzystywać wbudowane peryferium TAMP. Połączenie go z czujnikiem otwarcia obudowy pozwoliłoby na automatyczne kasowanie wrażliwych obszarów pamięci (rejestrów backupowych, kluczy wyprowadzonych w RAM) natychmiast po wykryciu fizycznej ingerencji, co udaremniłoby atak z użyciem debuggera.

2.5 Analiza ostatniej komunikacji (log TX) i podrobienie ramki

Na podstawie wykradzionego klucza zweryfikowano strukturę ramki LAVIET_FRAME_V1 dla wiadomości wysłanej przez urządzenie (payload 00 4A 00 00 00 37). System poprawnie wygenerował `frame_mac` przy użyciu klucza `hmac_key`, co oznacza, że dysponując kluczem można **tworzyć i podpisywać dowolne ramki** akceptowane przez sieć.

Potwierdzeniem jest log po stronie bramki, w którym wartości `expected_mac` i `received_mac` są identyczne dla wiadomości spoofowanej:

```
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
```

Zaobserwowano również nową, **zaszyfrowaną** transmisję (`msg=0x0128`, `counter=91`), wcześniej niewidzianą przez node STM32 — z innym tagiem MAC (3E 5C 81 52 ...) niż ramki jawne. Potwierdza to działanie licznika anti-replay (rosnący `counter`) oraz obecność ruchu szyfrowanego AES-CTR obok ramek jawnych z payloadem `qwerty`.


```
RADIO RX len=51 RSSI=-74 SNR=3
RADIO RX RAW: ....i...V.qwerty..).H.....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload
len=6 rssi=-74 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75
DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
RADIO RX len=51 RSSI=-75 SNR=3
RADIO RX RAW: ....i...V.qwerty..).H.....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload
len=6 rssi=-75 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75
DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
RADIO RX len=51 RSSI=-77 SNR=3
RADIO RX RAW: ....i...V.qwerty..).H.....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload
len=6 rssi=-77 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75
DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
```

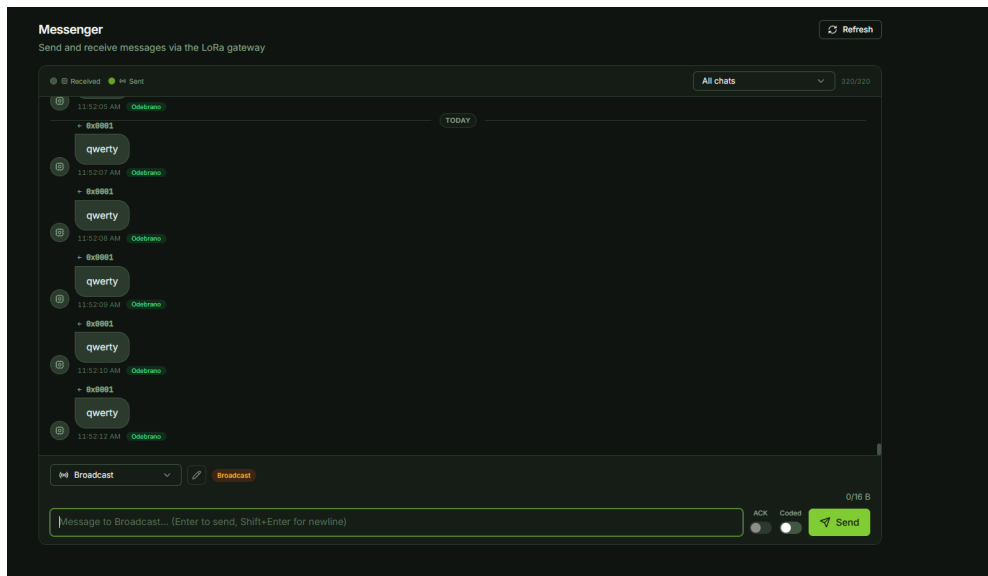
Rysunek 3: Logi RADIO RX po stronie node'a — ramki jawne (qwerty, msg=0x0069, dst=0x77CD), parsowanie nagłówka i HMAC failed/drop.

```

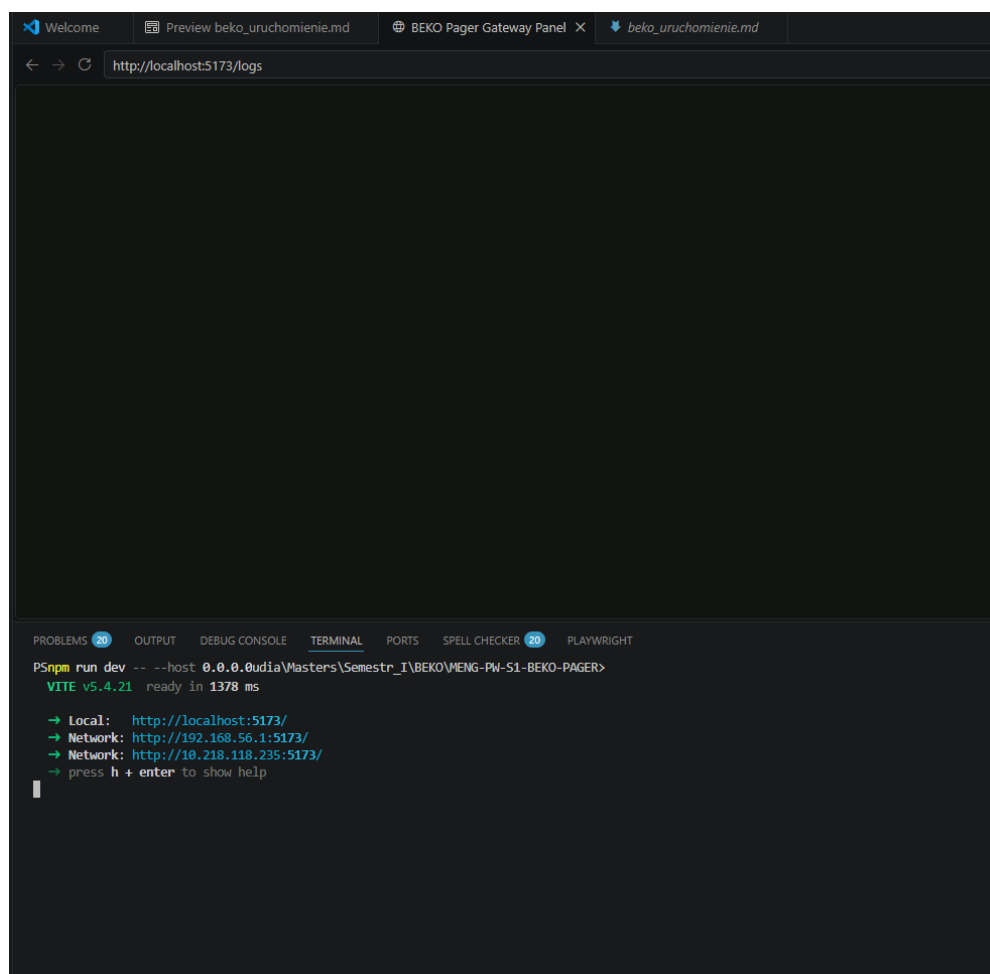
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0128
RADIO RX len=51 RSSI=-77 SNR=3
RADIO RX RAW: ....w..([qwerty>\R...5..f"...%.okn..Qp.\.^9g
RADIO RX HEX: 11 02 00 01 77 CD 01 28 00 00 00 5B 06 71 77 65 72 74 79 3E 5C 81 52 CD E2 5F 16 35 80 CD 66 22 02
B1 DE 25 87 82 6F 6B 6E EB DE 51 70 DF 5C D5 5E 39 67
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0128 counter=91 payload_len=6 rssi=-77 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: 3E 5C 81 52 CD E2 5F 16 35 80 CD 66 22 02 B1 DE 25 87 82 6F 6B 6E EB DE 51 70 DF 5C D5 5E 39 67
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0128
RADIO RX len=51 RSSI=-78 SNR=3
RADIO RX RAW: ....w..([qwerty>\R...5..f"...%.okn..Qp.\.^9g
RADIO RX HEX: 11 02 00 01 77 CD 01 28 00 00 00 5B 06 71 77 65 72 74 79 3E 5C 81 52 CD E2 5F 16 35 80 CD 66 22 02
B1 DE 25 87 82 6F 6B 6E EB DE 51 70 DF 5C D5 5E 39 67
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0128 counter=91 payload_len=6 rssi=-79 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: 3E 5C 81 52 CD E2 5F 16 35 80 CD 66 22 02 B1 DE 25 87 82 6F 6B 6E EB DE 51 70 DF 5C D5 5E 39 67
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0128

```

Rysunek 4: Logi RADIO RX — nowa zaszyfrowana ramka msg=0x0128, counter=91, odmienny tag MAC (payload pozostaje ciphertextem).



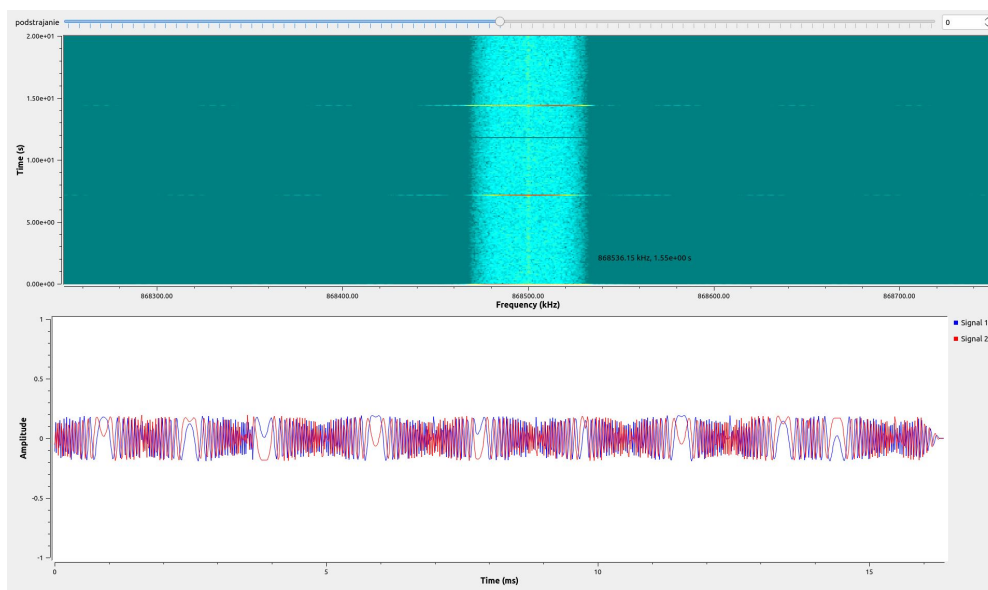
Rysunek 5: Panel Messenger bramki — strumień wiadomości broadcast/DATA (qwerty, 0x0001) wysyłanych i odbieranych przez sieć LoRa.



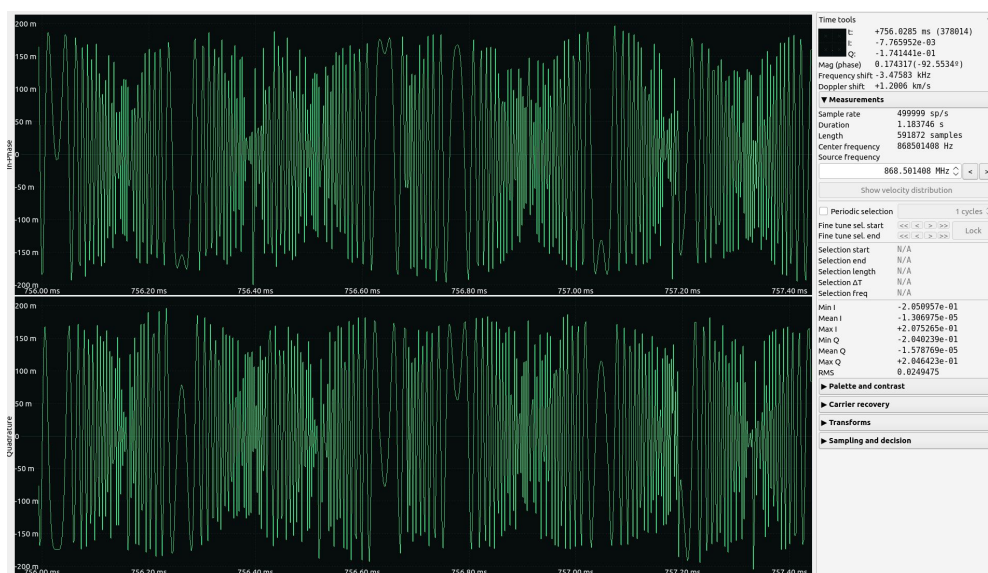
Rysunek 6: Webowy panel bramki BEKO (serwer dev Vite, localhost:5173) — interfejs zarządzania i podglądu logów gatewaya.

2.6 Analiza i ataki w warstwie radiowej (SDR)

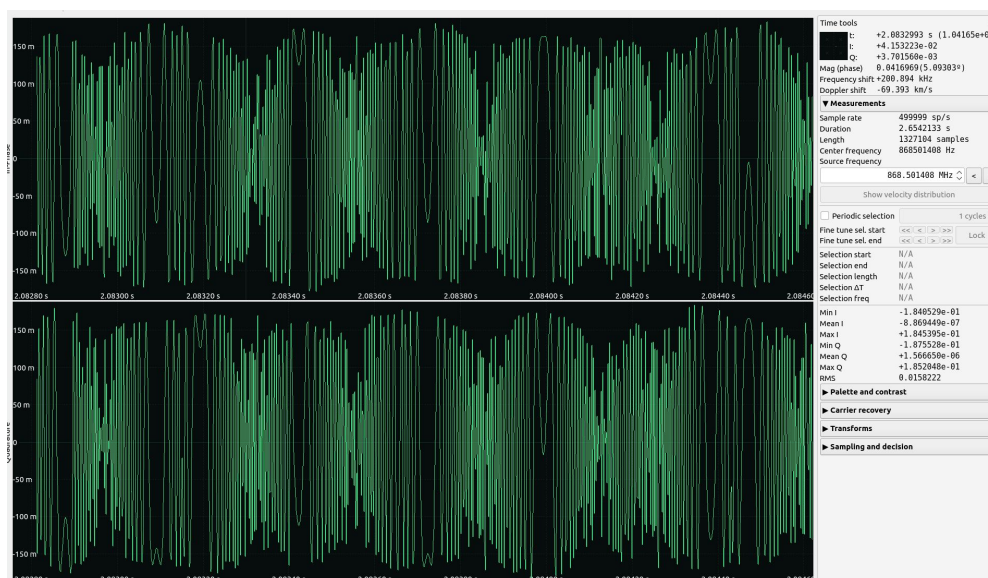
Z użyciem SDR oraz GNU Radio przeprowadzono akwizycję i analizę sygnału LoRa na częstotliwości nośnej **868.501408 MHz** (sample rate ok. 500 ksps). Przechwycono i przeanalizowano przebiegi I/Q transmisji, a następnie zbudowano flowgraph służący do generowania sygnałów zakłócających (jammer) z możliwością wyboru: szumu szerokopasmowego, szumu pasmowego, nośnej z przesunięciem oraz odtwarzania zarejestrowanego sygnału (replay z pliku `Replay-Waveform.raw`).



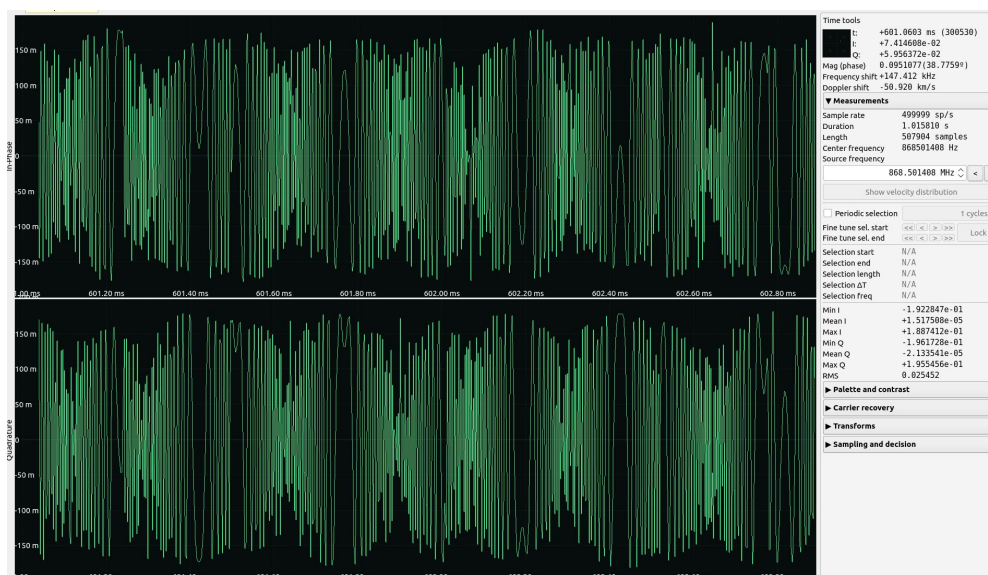
Rysunek 7: SigDigger — spektrogram w okolicy **868.5 MHz** (sygnał ok. 868536 kHz) oraz przebiegi czasowe Signal 1 / Signal 2.



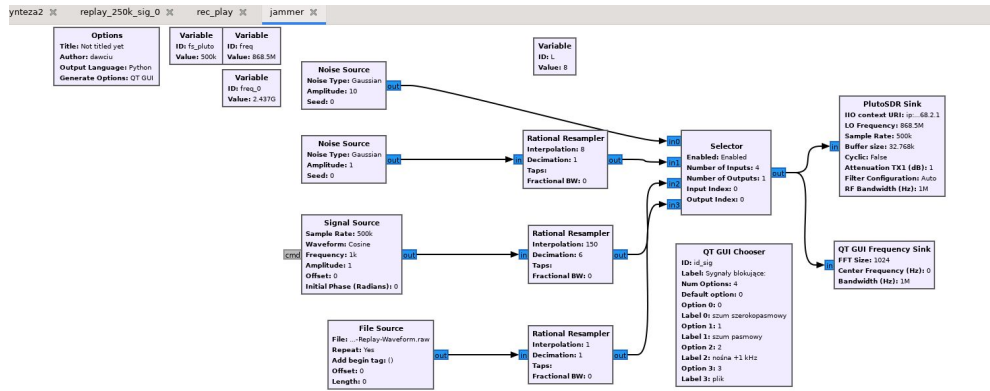
Rysunek 8: Akwizycja I/Q (próbka @756 ms): 499999 sp/s, center 868501408 Hz; pomiar przesunięcia częstotliwości / Dopplera.



Rysunek 9: Akwizycja I/Q (próbka @2.08 s): widoczne przesunięcie nośnej (+200 kHz) — ilustracja wpływu offsetu LO na demodulację.



Rysunek 10: Akwizycja I/Q (próbka @601 ms): +147 kHz frequency shift, RMS 0.025 — kolejny zarejestrowany fragment transmisji.



Rysunek 11: Flowgraph GNU Radio — generator sygnałów blokujących (jammer) z selektorem źródła (szum szeroko-/pasmowy, nośna +1 kHz, plik replay) i sinkiem PlutoSDR (LO 868.5 MHz).

Podsumowanie warstwy radiowej. System jest podatny na **pasywny podsłuch** i analizę transmisji oraz na **atak typu replay** zarejestrowanych ramek. Nie wykazano natomiast podatności na **zagłuszanie (jamming)** — dzięki zastosowaniu modulacji LoRa (rozpraszanie widma, CSS) zakłócanie nadajnikiem PlutoSDR nie spowodowało przerwania komunikacji między bramką Raspberry Pi a węzłem STM32. Skuteczny okazał się natomiast **wielokrotny replay**: powtarzane wysyłanie tych samych ramek zappełniło bufor i doprowadziło do zawieszenia serwera obsługującego komunikację (efekt zbliżony do ataku DoS).

2.7 Demodulacja przechwyconych ramek (LoRa/CSS)

Przechwycone nagrania IQ poddano demodulacji autorskim skryptem `demodulate.py`, który odtwarza warstwę fizyczną LoRa/CSS i pozwala zweryfikować obecność oraz strukturę transmisji. Pipeline przetwarza plik `.raw` z SigDigger (format `cf32`, interleaved float32 I/Q, $f_s = 500$ kHz, bez decymacji) w czterech etapach:

1. **Wczytanie próbek IQ** — para (I, Q) tworzy próbkę zespoloną $s[n] = I[n] + jQ[n]$; reprezentacja zespolona pozwala odróżnić częstotliwości powyżej i poniżej LO (868,5 MHz).
2. **Demodulacja CSS** — każdy symbol to liniowy chirp. Przy $SF = 7$ symbol trwa $T_{sym} = 2^{SF}/BW = 128/500\,000 = 256\ \mu s$, czyli dokładnie 128 próbek. Blok 128 próbek mnożony jest przez sprzężony bazowy up-chirp (de-chirp), a pozycja maksimum FFT wyznacza wartość symbolu $\hat{m} \in \{0, \dots, 127\}$.
3. **Pakowanie symboli w bajty** — 7 bitów na symbol, MSB first.
4. **Wyjście** — rzut hex zdekodowanych bloków.

Parametry demodulacji odpowiadają konfiguracji węzła: **SF7**, **BW 500 kHz**, **CR 4/5**, preambuła 8, sync word 0x34, CRC on, explicit header.

Zarejestrowany materiał. Zapisano przebiegi trzech transmisji sieci: broadcast (RPI-Broadcast-Waveform), node 0x4431 (RPI-Node-0x4431-Waveform) oraz node 0x77CD (RPI-Node-0x77CD-Waveform). Potwierdza to możliwość pasywnej akwizycji i odtworzenia (replay) ruchu radiowego w sieci.

Ograniczenia analizy. Implementacja celowo pomija dekodery FEC (Hamming), Gray decode oraz detekcję preambuły i sync word, a praca przy $f_s = BW$ nie zostawia marginesu na offset LO — przesunięcie nośnej o kilkadziesiąt herców względem 868,5 MHz przekłada się na błąd pozycji prążka FFT, a więc przekłamanie wartości symbolu. Celem etapu była weryfikacja obecności i struktury transmisji, a nie produkcyjna demodulacja całego stosu LoRa.

3. Rekomendacje naprawcze

Na podstawie zidentyfikowanych podatności rekomenduje się następujące działania naprawcze dla twórców systemu:

- **Wyłączenie komunikatów debugowych na UART** w kompilacji produkcyjnej (Release) — eliminacja wycieku `hmac_key` oraz `pair_code`.
- **Sprzętowa blokada odczytu pamięci RDP na Poziom 1** (docelowo Level 2) w STM32 — uniemożliwienie odczytu FLASH i dynamicznego zrzutu RAM przez debugger.
- **Szyfrowanie nośnika (LUKS)** na bramce Raspberry Pi — ochrona przed przejęciem systemu z poziomu karty SD.
- **Wdrożenie Secure Boot** na node STM32 — zapobieżenie wgraniu złośliwego firmware przy fizycznym dostępie do urządzenia.
- **Aktywacja peryferium TAMP** z czujnikiem otwarcia obudowy — kasowanie sekretów po wykryciu ingerencji (anti-tamper).
- **Dokończenie rotacji kluczy** (radiowy KEY_ROTATE oraz wymiana DH/ECDH) — odejście od statycznego, współdzielonego klucza HMAC.

4. Propozycja implementacji RFID

Jako uzupełnienie kontroli dostępu fizycznego (rola **Serwisant** i lokalny PIN node'a) proponuje się dodanie modułu **RFID/NFC** (np. czytnik MFRC522, 13.56 MHz, interfejs SPI) do node'a STM32 oraz/lub do bramki. Karta lub token zbliżeniowy zastępowałby ręczne wpisywanie PIN-u przy dostępie do trybu konfiguracji i czynności serwisowych, podnosząc wygodę i rozliczalność (logowanie, kto i kiedy uzyskał dostęp).

Kluczowe założenia bezpieczeństwa: aby uniknąć trywialnego klonowania kart, należy stosować karty z uwierzytelnianiem **challenge-response** (np. MIFARE DESFire EV2/EV3 zamiast UID-only). UID karty **nie może** być jedynym sekretem — identyfikator karty powinien być powiązany z materiałem chronionym przez **TPM** (analogicznie do root seeda), a sama autoryzacja realizowana kryptograficznie, nie przez proste porównanie UID. Listę uprawnionych tokenów należy przechowywać w *trusted storage* (jak sloty zaufanych urządzeń), a każde użycie logować na UART. Takie rozwiązanie zachowuje spójność z istniejącą architekturą zaufania systemu (TPM + HMAC) i nie wprowadza nowego, słabszego punktu uwierzytelniania.